

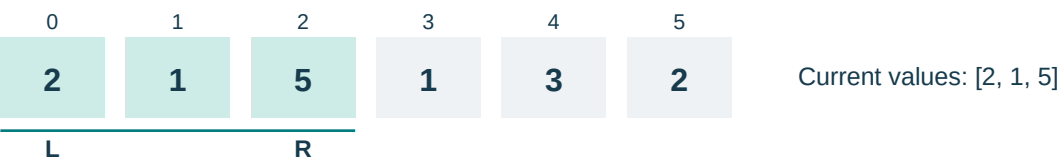
Sliding windows, explained

Your Day 3 tutorial is complete. These notes include the examples, corrections, pointer diagrams, and runtime explanations from our discussion. Use them to understand each move, then practise writing the code without looking.

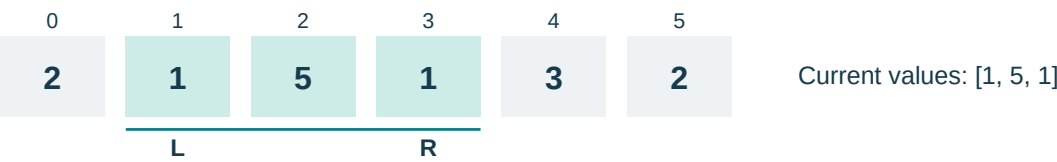
A window is a continuous section

In an array we call it a subarray; in a string we call it a substring. Elements must occupy adjacent positions. You cannot skip a value in the middle and still call the result a window.

A size-3 window



Slide one position right



The shaded cells belong to the window. The small numbers above them are zero-based indexes; L and R mark the included endpoints. Window length is right - left + 1.

Pattern	Goal	Main action
Fixed size	Largest sum of exactly k values	Remove outgoing; add incoming
Shortest valid window	Smallest length with sum >= target	Save a valid answer, then shrink
Longest valid window	Most characters without repeats	Fix duplicates, then save the answer

How to use this guide

Pages 2-3: fixed windows. Pages 4-6: minimum-sum windows and why L moves. Page 7: the nested-loop O(n) proof and worst-case counts. Pages 8-9: strings and sets. Pages 10-11: pattern comparison, clickable practice links, and self-checks.

Assumptions: k is an integer; the minimum-length sum algorithm uses strictly positive numbers and a positive target. Fixed-size sum windows can include negatives. The string algorithm uses average O(1) set operations.

1. Fixed-size maximum sum

Find the largest sum of exactly k consecutive values. Consecutive means adjacent: for $[2, 1, 5, 1, 3, 2]$ and $k = 3$, the best window is $[5, 1, 3]$, with sum 9.

```
def max_window_sum(nums, k):
    if k <= 0 or k > len(nums):
        return None

    window_sum = 0
    for i in range(k):
        window_sum += nums[i]

    best_sum = window_sum
    for right in range(k, len(nums)):
        window_sum -= nums[right - k]
        window_sum += nums[right]
        best_sum = max(best_sum, window_sum)

    return best_sum
```

Read the syntax in execution order

len(nums) gives the number of elements. **or** means either invalid-size condition is enough to return None. **range(k)** generates indexes 0 through $k - 1$; its stop value is excluded.

+= adds to a variable; **-=** subtracts from it. After building the first window, **best_sum = window_sum** records its actual sum. Starting **best_sum** at 0 would give a wrong answer when all window sums are negative.

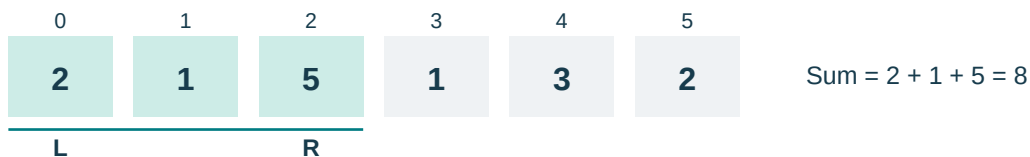
right is the incoming index. The outgoing index is **right - k**, exactly k positions behind it. **max(a, b)** keeps the larger value. Return **best_sum**, because the last window need not be the best one.

```
print(max_window_sum([2, 1, 5, 1, 3, 2], 3)) # 9
print(max_window_sum([-5, -2, -8], 2))      # -7
print(max_window_sum([1, 2], 3))           # None
```

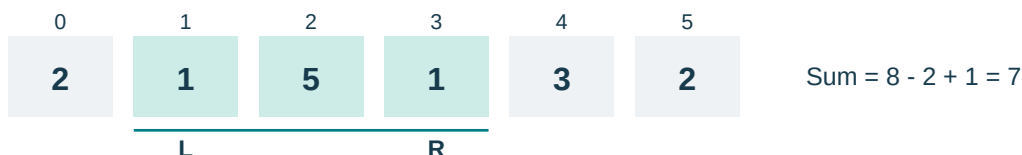
Cost: k initial additions plus $n - k$ slides gives $O(n)$ time. Only a few variables are kept: $O(1)$ auxiliary space. Recalculating every window from scratch would take $O((n - k + 1) * k)$ time.

2. What leaves, what enters?

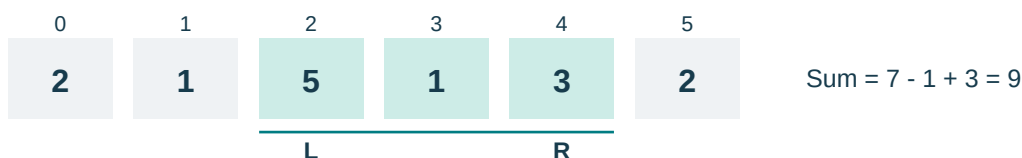
First window



Next window



Next window



For [1, 5, 1] becoming [5, 1, 3], the outgoing value is the first 1; the incoming value is 3. The 5 is already present and stays inside the window, so adding 5 again would count it twice.

Incoming index	Outgoing index	Update	Current / best sum
3	0	$8 - 2 + 1$	7 / 8
4	1	$7 - 1 + 3$	9 / 9
5	2	$9 - 5 + 2$	6 / 9

The arithmetic we practised

```
nums = [3, 1, 4, 2]
k = 2
# [3, 1] -> 4
# [1, 4] -> 5
# [4, 2] -> 6
# Maximum sum: 6
```

Another slide: [2, 5] has sum 7. To become [5, 1], remove 2 and add 1: $7 - 2 + 1 = 6$. Keep the shared 5; do not add it again.

3. Shortest window reaching a target

Find the shortest subarray whose sum is at least target. For [2, 3, 1, 2, 4, 3] and target 7, [4, 3] qualifies with length 2. Return the length, not the sum or the subarray.

```
def min_window_length(nums, target):
    left = 0
    window_sum = 0
    best_length = len(nums) + 1

    for right in range(len(nums)):
        window_sum += nums[right]

        while window_sum >= target:
            length = right - left + 1
            best_length = min(best_length, length)
            window_sum -= nums[left]
            left += 1

    if best_length == len(nums) + 1:
        return 0
    return best_length
```

What each variable means

left is the first included index; **right** is the last. **window_sum** belongs only to the current window. **best_length** remembers the shortest valid length found anywhere so far.

len(nums) + 1 is an impossible answer, used to mean "not found yet." **>=** means greater than or equal to. **min** chooses the smaller length. The extra 1 in `right - left + 1` counts both endpoints: indexes 2 through 4 contain three elements.

The while loop saves the valid answer **before removing** a number. Then it removes `nums[left]` and advances `left`. Removing first is essential: after incrementing `left`, it would point at a different number.

```
print(min_window_length([2, 3, 1, 2, 4, 3], 7)) # 2
print(min_window_length([2, 5, 1], 5))         # 1
print(min_window_length([1, 1, 1], 7))         # 0
```

Positive-input requirement: expansion increases the sum, and shrinking decreases it. Negative values break that reasoning. A nonpositive target is also outside this function's contract.

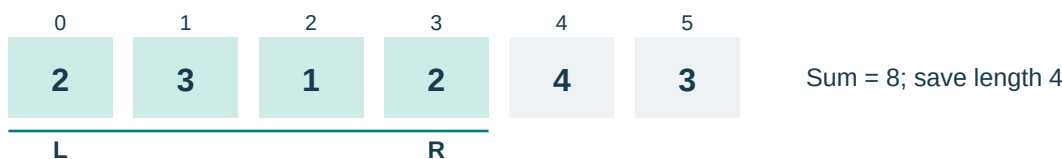
4. Why does L move forward?

L moves to try making a valid window shorter. At sum 6, do not shrink yet: first add another number from the right. Once a window qualifies, save its length and test whether dropping its first value still works.

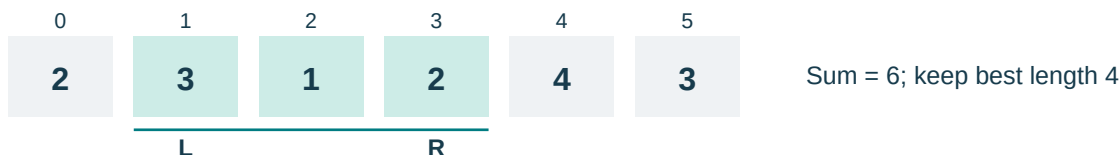
A. Still too small



B. Add 2 at R = 3



C. Remove 2 at L = 0; move L



D. Add 4 at R = 4



Why can we leave the first 2 behind? We already recorded a qualifying window starting there. Any future window with that same start and a farther right endpoint would be longer, so it cannot improve our shortest answer.

Do not move L backward. A failed shrink does not erase the answer already saved. Keep the smaller current window and expand it next. Current validity and best answer are separate pieces of information.

```
# While the current sum is valid:
best_length = min(best_length, right - left + 1)
window_sum -= nums[left] # Remove the old leftmost value
left += 1                # Then advance the boundary
```

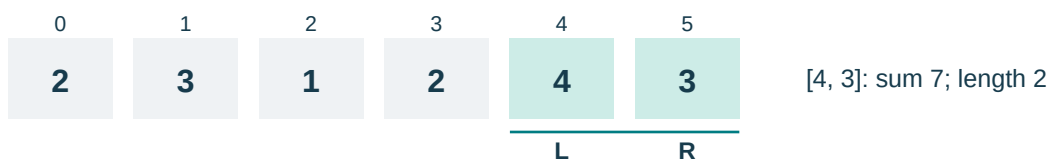
5. Follow every add and removal

Input [2, 3, 1, 2, 4, 3], target 7. A valid window has sum at least 7. The answer column changes only when a shorter valid window appears.

Action	L / R	Current window	Sum	Best length
Add 2	0 / 0	[2]	2	Not found
Add 3	0 / 1	[2, 3]	5	Not found
Add 1	0 / 2	[2, 3, 1]	6	Not found
Add 2; save	0 / 3	[2, 3, 1, 2]	8	4
Remove 2	1 / 3	[3, 1, 2]	6	4
Add 4; save	1 / 4	[3, 1, 2, 4]	10	4
Remove 3; save	2 / 4	[1, 2, 4]	7	3
Remove 1	3 / 4	[2, 4]	6	3
Add 3; save	3 / 5	[2, 4, 3]	9	3
Remove 2; save	4 / 5	[4, 3]	7	2
Remove 4	5 / 5	[3]	3	2

Rows marked "save" include the next valid-window check. In the code, updating the answer happens at the start of a while iteration. After removing 3, the sum remains 7, so the loop repeats and saves length 3 before removing 1.

Shortest valid window found



At the end, the current window is [3], which does not qualify. That is fine: `best_length` still contains 2. We return the saved answer, not the length of the final window.

6. Nested loops can still be $O(n)$

Count how much work the inner loop performs over the entire function. It does not start a full scan again for each outer iteration.

```
# Quadratic example: resets left on every outer iteration
for right in range(n):
    left = 0
    while left < n:
        left += 1
# n outer iterations * n inner iterations = n squared

# Sliding window: left is initialized only once
left = 0
for right in range(len(nums)):
    window_sum += nums[right]
    while window_sum >= target:
        window_sum -= nums[left]
        left += 1
```

In the second sketch, assume `window_sum` starts at 0 and the inputs satisfy the positive-number contract. Every inner iteration advances `left`. Since `left` never resets or moves backward, it can advance at most n times overall. Each element enters once and leaves at most once.

Worst case: [5, 5, 5, 5], target = 5

Each addition immediately makes the window valid. The inner loop removes that same 5 and stops. With $n = 4$:

What is counted	Example count	General bound
Outer-body executions	4	n
Inner-body executions, total	4	At most n
Combined body executions	8	At most $2n$
While-condition checks	8	$n + q$, where $q \leq n$

There is one successful while check for each of the q inner iterations, plus one final failed check for each outer iteration. This counts checks, not every Python operation. Adding all constant-time assignments, comparisons and updates still gives a constant multiple of n : $O(n)$.

For 1,000 elements, at most 1,000 additions and 1,000 removals occur. A single outer iteration may perform many removals, but that spends moves that cannot be repeated later. This reasoning depends on bounded forward movement and constant work per iteration.

7. Longest substring without repeats

Now the goal is the longest valid window. A set stores the characters already inside it. Before adding a repeated character, remove from the left until the older copy is gone.

```
def longest_unique_substring(text):
    seen = set()
    left = 0
    best_length = 0

    for right in range(len(text)):
        while text[right] in seen:
            seen.remove(text[left])
            left += 1

        seen.add(text[right])
        length = right - left + 1
        best_length = max(best_length, length)

    return best_length
```

Syntax and ordering

set() creates an empty set; **{}** would create a dictionary. **in** checks membership. **remove** deletes an existing character, and **add** inserts one. The window contains unique characters, so removing the outgoing character updates the set correctly.

During the while loop, the incoming character has **not yet been added**. We first remove its older occurrence, then add the new one. Save the longest length only after the window has been made valid.

```
print(longest_unique_substring("abba"))    # 2
print(longest_unique_substring("pwwkew")) # 3
print(longest_unique_substring("aaaa"))    # 1
print(longest_unique_substring("abca"))    # 3
print(longest_unique_substring(""))        # 0
```

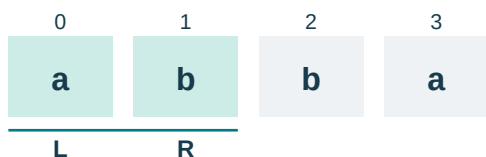
Return type: for "aaaa", "a" is a valid answer substring, but this function returns its length, 1. For "abca", "abc" and "bca" both have length 3.

Cost: $O(n)$ average time using average $O(1)$ set operations. Extra space is $O(\min(n, A))$, where A is the number of possible distinct characters. Character case, spaces, and punctuation are treated as ordinary characters.

8. Why removing once is not enough

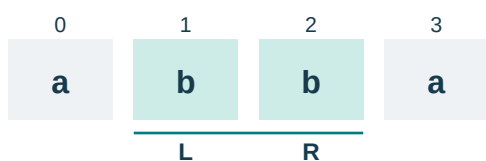
For "abba", adding the second b would make the candidate window "abb". Removing a leaves "bb", which still repeats. We must remove the older b as well. This is why the condition is a while loop, not an if.

A. Valid window before incoming b



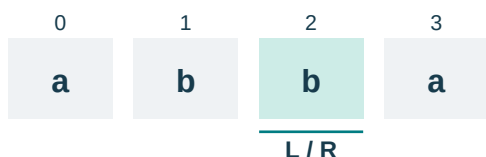
Window "ab"; seen = {a, b}

B. Remove a from the candidate



Candidate "bb": still repeats

C. Remove old b; add incoming b



Window "b"; seen = {b}

The middle picture shows the candidate including the incoming b. In the code, seen contains only the old b at that moment; the new b is added after the while loop. The set cannot store two copies of b, so do not add it first.

Trace "pwwkew"

Incoming	Removal before insertion	Valid window	Best
p	None	p	1
First w	None	pw	2
Second w	Remove p, then old w	w	2
k	None	wk	2
e	None	wke	3
Final w	Remove old w	kew	3

"pwke" is not a substring of "pwwkew": it requires skipping a character. A substring uses consecutive positions. "wke" and "kew" are both valid length-3 answers.

9. Choose when to move and save

Problem	When L moves	When to save
Fixed-size maximum sum	One step per slide after the first window	After outgoing removal and incoming addition
Shortest sum $\geq$ target	While sum qualifies; try shortening	Before removal, while valid
Longest unique substring	While incoming character already exists	After duplicate removal and insertion

Only valid windows can improve the answer

```
# Shortest: record BEFORE shrinking a valid window.
while window_sum >= target:
    best_length = min(best_length, right - left + 1)
    window_sum -= nums[left]
    left += 1

# Longest: repair first, then record the valid window.
while text[right] in seen:
    seen.remove(text[left])
    left += 1
seen.add(text[right])
best_length = max(best_length, right - left + 1)
```

Common mistakes and their fixes

Adding a value that stayed inside: shared elements are already included in the sum. Add only the incoming element.

Incrementing L before subtracting: subtract `nums[left]` first, so you remove the correct outgoing value.

Resetting L for every R: this repeats work and loses the linear-time behavior.

Returning the final window: return the saved best length or sum.

Using a set as a duplicate counter: it stores unique values, not frequencies; the string algorithm maintains a unique window before insertion.

Explain your solution in an interview

State whether the window is fixed or variable. Define what makes it valid and what each pointer means. Explain what enters, what leaves, and when you update the best answer. Trace a small example; then justify the total pointer movement and the input assumptions.

10. LeetCode practice and revision

These three official problem links match the patterns taught today. Start with the fixed-window question. Write the logic yourself before consulting these notes. Use the supplied Python 3 method names and parameters on LeetCode.

643. Maximum Average Subarray I - Easy

Find the largest average over exactly k consecutive elements. Our maximum-sum function gives the needed sum. Because all windows have the same length k , divide the best sum by k to get the maximum average.

```
# Adapt the final return inside the fixed-window solution:
return best_sum / k # / is ordinary division, not //
```

209. Minimum Size Subarray Sum - Medium

Use the positive-number shrinking window. Return the smallest qualifying length, or 0 if no window qualifies. Watch the argument order in the supplied `minSubArrayLen(self, target, nums)` method.

3. Longest Substring Without Repeating Characters - Medium

Return the length, not the substring. Use the set-based window from these notes. If the supplied input parameter is named `s`, use `s` consistently where the teaching function uses `text`.

Your completed checkpoint answers

Question	Answer
Slide [1, 5, 1] to [5, 1, 3]	Remove 1, add 3; new sum 9
Slide [2, 5] to [5, 1]	$7 - 2 + 1 = 6$
[3, 1, 4, 2], size 2	Window sums 4, 5, 6; maximum 6
Remove a from abb	bb still repeats; remove old b too
Longest unique window in aaaa	Substring a; returned length 1
Longest unique length in abca	3

Mastery checklist

- [] I can identify outgoing and incoming elements.
- [] I can explain why `L` moves and never needs to reset.
- [] I can distinguish the current window from the saved best answer.
- [] I can justify at most n additions and n removals.
- [] I can explain where each algorithm updates its answer.
- [] I can trace empty strings, negative fixed-window sums, and no-solution positive arrays.

Day 3 tutorial complete. Revisit the diagrams whenever a pointer movement feels unclear. The next day's PDF will follow completion of its interactive tutorial.